Backward Search

Backward search

- Backward versions of any of the forward search algorithms can be made.
- For many planning problems, the branching factor is large when starting from x_I
  - In this case, it might be more efficient to start the search at a goal state and work backward until the initial state is envountered.

o Backward version of Dijkstra's algorithm.

Backward search

Forward approach

$f: X \times U \rightarrow X$

$f(x, u) = x'$

U: set of possible actions $= \{u_i\}$

$U^{-1} = \{(x, u) \in X \times U \mid x \in X, u \in U\}$

$U^{-1}(x') = \{(x, u) \in U^{-1} \mid x' = f(x, u)\}$

- It may be preferable to pre-compute a representation of transition function f for the "backward" search algorithm.
  - Backward action space
  - Backward action space
  - Perfect symmetry with respect to the forward search

From a given $x' \in X$, the set of all $(x, u) \in U^{-1}$ that map to x' using f.

Backward search

Backward - Dijkstra (G, s, g)

for each vertex v in G:

dist [v] = ∞ ; parent [v] : null.

dist [g] = 0 ✓

Q = Priority Queue

Insert (g, 0) into Q.

while (Q is not empty)

u = vertex in Q with minimum dist

remove u from Q.

if u == s

success; break;

– continue to the next page —

Backward search

for each predecessor v of u:

new_dist = dist[u] + cost(v, u)

if new_dist < dist[v]

dist[v] = new_dist

parent[v] = u

update v in Q.

return dist[s], parent.

Backward search algorithm

```
BACKWARD_SEARCH
 1   Q.Insert(x_G) and mark x_G as visited
 2   while Q not empty do
 3       x' ← Q.GetFirst()
 4       if x = x_I
 5           return SUCCESS
 6       forall u^-1 ∈ U^-1(x)
 7           x ← f^-1(x', u^-1)
 8           if x not visited
 9               Mark x as visited
10               Q.Insert(x)
11           else
12               Resolve duplicate x
13   return FAILURE
```

Bidirectional search

- Combines the concepts of *forward* and *backward* search.
  - One tree is grown from the initial state, and the other is grown from the goal state.
  - The search terminates with **success** when the two trees meet
  - When does the search **fails** ?
- Bidirectional search can dramatically reduce the amount of required exploration

Bidirectional search algorithm

```
BIDIRECTIONAL_SEARCH
 1   Q_I.Insert(x_I) and mark x_I as visited
 2   Q_G.Insert(x_G) and mark x_G as visited
 3   while Q_I not empty and Q_G not empty do
 4       if Q_I not empty
 5           x ← Q_I.GetFirst()
 6           if x already visited from x_G
 7               return SUCCESS
 8           forall u ∈ U(x)
 9               x' ← f(x, u)
10               if x' not visited
11                   Mark x' as visited
12                   Q_I.Insert(x')
13               else
14                   Resolve duplicate x'
```

Bidirectional search algorithm (contd.)

```
15      if Q_G not empty
16          x' ← Q_G.GetFirst()
17          if x' already visited from x_I
18              return SUCCESS
19          forall u^{-1} ∈ U^{-1}(x')
20              x ← f^{-1}(x', u^{-1})
21              if x not visited
22                  Mark x as visited
23                  Q_G.Insert(x)
24              else
25                  Resolve duplicate x
26  return FAILURE
```

Bidirectional search

Remember, $x_I \in X$ and $X_G \subset X$

- Dijkstra and A* variants of bidirectional search lead to optimal solutions

Thank you